

Phlower: A Deep Learning Framework Supporting PyTorch Tensors with Physical Dimension

Riku Sakamoto
RICOS Co. Ltd.



INTRODUCTION

- Machine learning for physical simulations has recently attracted attention in the scientific computing community.
- Table 1 shows that physical dimensions are the fundamental quantities used to describe physical systems.
- In numerical simulations of physical phenomena, maintaining physical dimensional consistency is essential to ensure the correctness of computations, as illustrated in Figure 1. However, few tools support this during deep learning computations.

$$\underbrace{(\mathbf{u} \cdot \nabla) \mathbf{u}}_{LT^{-2}} = - \underbrace{\frac{1}{\rho} \nabla p}_{LT^{-2}} + \underbrace{\nu \nabla^2 \mathbf{u}}_{LT^{-2}}$$

Figure 1. Example of dimension analysis: Steady incompressible Navier–Stokes momentum equation. All terms have the same dimension of acceleration. Here, $\mathbf{u}$ is the velocity field. p is pressure ρ is density ν is the kinematic viscosity

OBJECTIVE

Phlower integrates physical dimensional information into PyTorch tensor objects. **By enforcing dimensional consistency, Phlower enables users to avoid errors and improve the reliability** of their deep learning models.

Table 1. Physical Dimensions	
Quantity	Symbol
Time	T
Mass	M
Length	L
Electric current	I
Thermodynamic temperature	Θ
Amount of substance	N
Luminous intensity	J

Phlower keeps your computations dimensionally correct

```
from phlower import phlower_tensor
import torch

mass = phlower_tensor(
    torch.rand(10), dimension={"M": 1}
)
velocity = phlower_tensor(
    torch.rand(10), dimension={"L": 1, "T": -1}
)

# 1/2 m v^2
kinetic_energy = 0.5 * mass * velocity ** 2
print(kinetic_energy.dimension)
# Output: PhlowerDimensionTensor(T: -2.0, L: 2.0, M: 1.0,
I: 0.0, Theta: 0.0, N: 0.0, J: 0.0)
```

CORE CONCEPT

Dimensional Consistency

Phlower enforces dimensional consistency during tensor operations, ensuring that **operations involving tensors with incompatible physical dimensions raise informative errors**.

Seamless Integration

PhlowerTensor is **compatible with PyTorch's autograd system and core tensor operations**.

Lightweight Implementation

Figure 2 illustrates that PhlowerTensor is implemented as a lightweight wrapper around PyTorch tensors. This design allows users to **seamlessly convert between PyTorch tensors and PhlowerTensor with minimal performance overhead**.

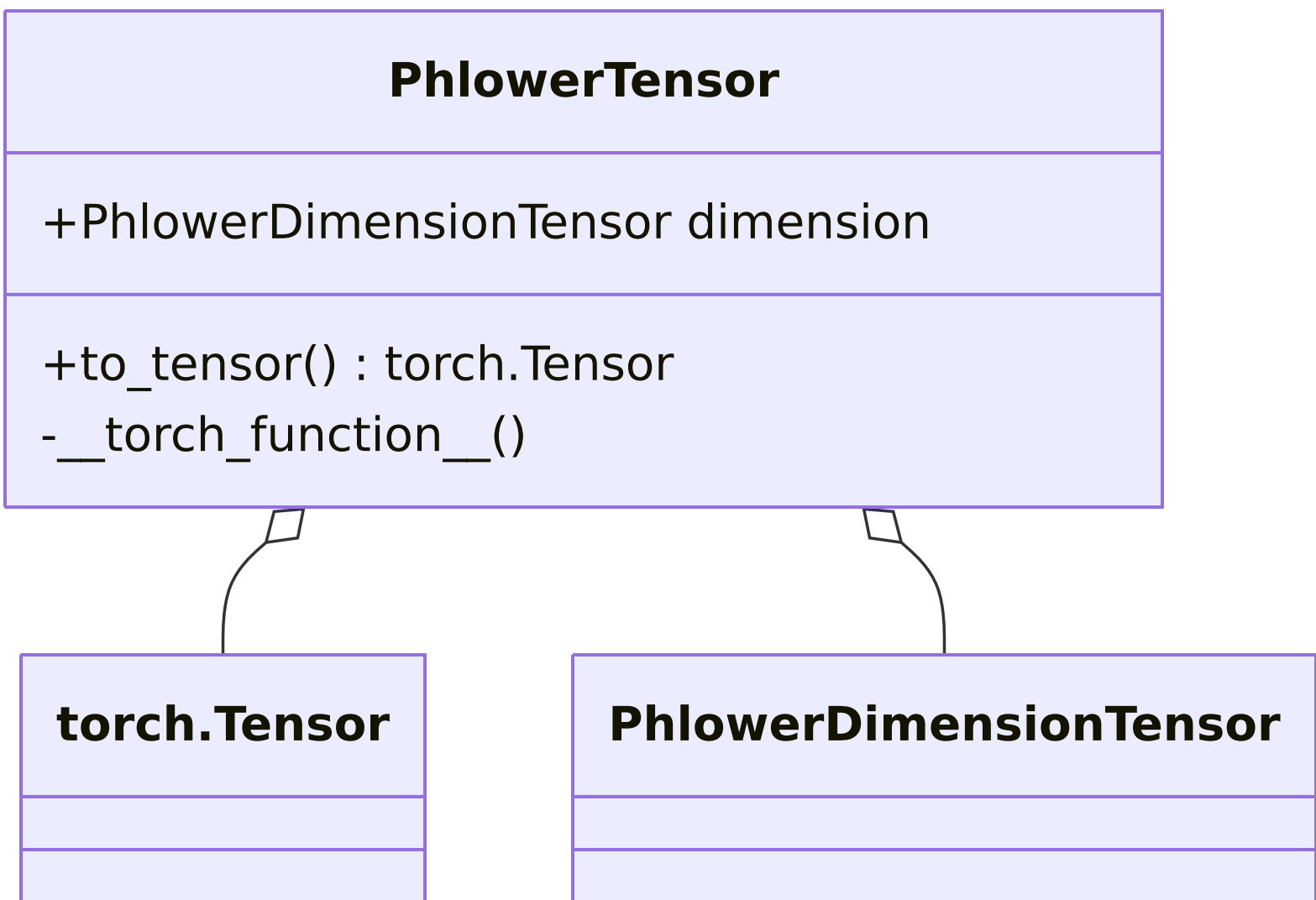


Figure 2. Class diagram of PhlowerTensor

OTHER FEATURES

Shape Semantics

Each index in the shape of PhlowerTensor carries semantic meaning. For example, when you define time-series scalar data, the first index corresponds to time steps.

YAML-Based Model Definition

Phlower provides a YAML-based model definition system that allows users to define machine learning models. Figure 3 is an example of a Mermaid diagram exported from a defined model.

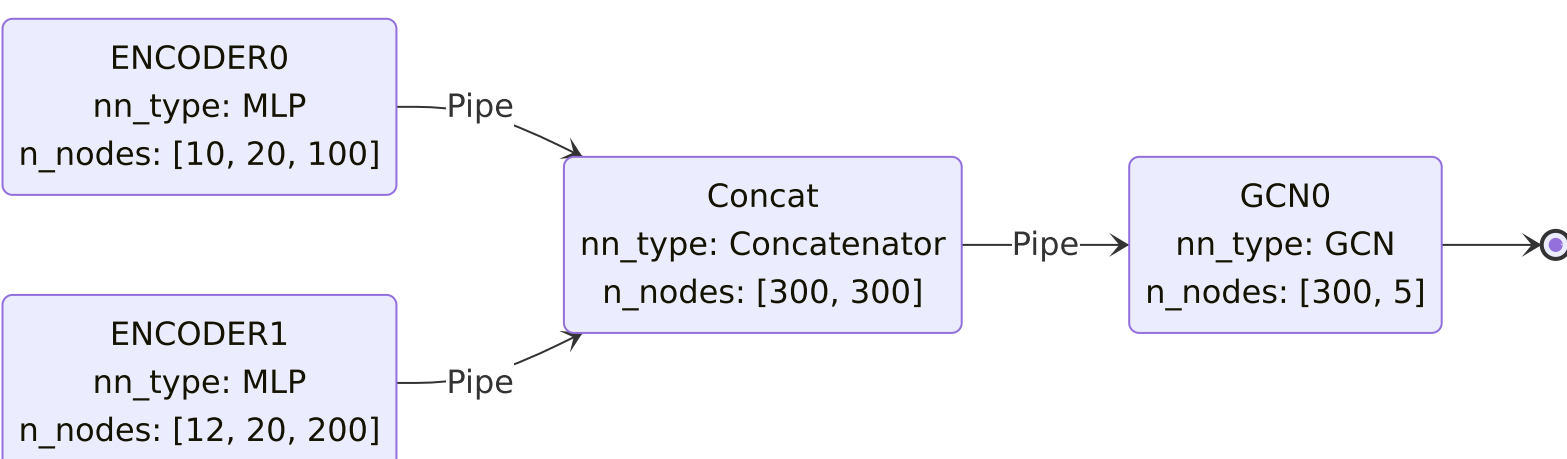


Figure 3. Example output of a model architecture diagram.

Limitations & Future Work

- ### Runtime Overhead
- Dimensional consistency is enforced at runtime, which introduces additional computational overhead.
- ### Partial Compatibility with PyTorch
- Certain advanced features such as in-place operations or custom autograd functions may require manual adaptation or may not be fully supported.



Visit the Phlower repository